P06 – Placement Skill Matcher and Readiness Analyser

1. Project Title Evidence-Based Placement Readiness and Skill-Gap Platform. *Track J:* Next.js + NestJS + MongoDB. *Track P:* Django + DRF + PostgreSQL.

2. Project Overview *Description.* Builds a verified skill profile for each student from academic results, project work, assessments, mock interviews and external evidence; models recruitment drives and their requirements; computes readiness and specific gaps per drive; converts gaps into a time-bound preparation plan; and gives the Placement Cell a batch-level readiness view before a drive opens. *Problem.* Placement preparation is driven by CGPA and generic training. A student can be formally eligible for a drive and still be unprepared, and nobody knows which students or which skills until the drive fails. *Target users.* Students, faculty, placement officers, placement head, recruiters (restricted), administrators. *Expected outcome.* Skill gaps identified weeks before a drive, targeted training instead of generic training, and a defensible batch readiness picture.

> **Critical design distinction: eligibility and readiness are separate.** A student may be eligible and unready, or ready and ineligible. The data model must keep these as independent computations, and the interface must never conflate them.

3. Project Objectives

- Maintain an institutional skill taxonomy with synonyms and levels
- Build student skill profiles from multiple evidence sources with verification status
- Model drives, roles and requirements
- Compute eligibility and readiness independently, with explanations
- Produce ranked, actionable gaps and a preparation plan
- Provide controlled recruiter access limited to registered students in an approved drive
- Provide batch and cohort readiness analytics to the Placement Cell

4. Key Features Skill taxonomy with levels and equivalences · multi-source evidence ingestion · verified and unverified evidence distinction · mock-interview and assessment capture · readiness scoring by dimension – technical, aptitude, communication, project depth · drive and eligibility management · per-drive gap analysis · preparation planner with schedule · recruiter portal with scoped visibility · batch readiness dashboards.

5. Functional Modules

#	Module	Function	Independent?
M1	Skill Taxonomy	Skills, levels, synonyms, clusters, role mappings	**Yes** – build first
M2	Evidence Ingestion	Academic, project, certification, assessment and external evidence	**Yes**
M3	Evidence Verification	Faculty verification workflow; verified and unverified flags	Depends on M2
M4	Student Skill Profile	Aggregates evidence into levelled skill claims	Depends on M1, M2, M3
M5	Assessment & Mock Interview	Rubric-based evaluation capture and history	**Yes**
M6	Readiness Engine	Dimension-wise readiness with explanation	Depends on M4, M5
M7	Drive & Eligibility Management	Drives, roles, criteria, registration, shortlists	**Yes**
M8	Match & Gap Analysis	Profile against drive requirement comparison	Depends on M4, M7
M9	Preparation Planning	Gap to time-bound plan with resources and checkpoints	Depends on M8
M10	Recruiter Portal & Batch Reporting	Scoped recruiter search; institutional analytics	Depends on M7

6. User Roles & Permissions **Student** – own profile, readiness, gaps, evidence and preparation plan; **cannot**

self-verify evidence. **Faculty** – record mock-interview evaluations; verify project and skill claims for own students. **Placement Officer** – drives, criteria, shortlists, batch reports, student detail. **Placement Head** – all of the above plus recruiter access control and institutional analytics. **Recruiter** – **only** students registered for their own approved drive, only fields approved for release, rate-limited, fully logged, **no bulk export**. **Administrator** – skill framework, evidence rules, scoring configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. **NestJS** for the domain complexity. **Meilisearch or Atlas Search** for skill and candidate search. The recruiter portal is a separate Next.js route group with its own session policy and stricter rate limits.

TRACK P · Python + Django + PostgreSQL. **PostgreSQL full-text search** (`SearchVector`, `SearchRank`) with `pg_trgm` for fuzzy skill names – reach for Meilisearch only if that proves insufficient. The recruiter portal is a **separate Django app with its own URL namespace, its own DRF permission classes and its own throttle rates**; DRF's built-in throttling does this well. Batch readiness runs as a Celery task. **Path P2** is reasonable here because the student dashboard is genuinely interactive.

8. Technical Requirements (deltas) *Frontend (both):* radar visualisation of readiness dimensions; gap list ordered by impact; preparation plan as a calendar; recruiter search with faceting. *Backend (both):* readiness recomputation triggered by new evidence, debounced; batch readiness computation for a 5,000-student cohort as a background job; skill matching using taxonomy expansion, so a student with "React" matches a requirement for "frontend framework". *Data model (both):* Skill, SkillSynonym, SkillCluster, Student, EvidenceItem, Verification, Assessment, MockInterview, SkillProfile, ReadinessSnapshot, Drive, DriveRequirement, EligibilityRule, Registration, GapAnalysis, PreparationPlan, RecruiterAccessGrant, AccessLog. *API (both):* `/skills/*`, `/students/:id/profile`, `/students/:id/readiness`, `/drives/*`, `/drives/:id/match`, `/preparation-plans`, `/recruiter/*`. *Security (both):* recruiter access is the highest-risk surface in this project – time-boxed grants, per-drive scoping, field-level release control, full access logging, no export endpoint, and an explicit negative test proving a recruiter cannot reach an unregistered student.

TRACK J · Node.js + MongoDB. Taxonomy expansion at query time through the search index; guard clauses in a dedicated authorisation module rather than scattered through controllers.

TRACK P · Python + Django + PostgreSQL. The recruiter's `get_queryset` filters to the approved drive *before* any other logic runs, and the serializer exposes only released fields – two independent layers, either of which alone would be sufficient and neither of which you rely on alone. Access grants carry an expiry checked in the permission class, not in a template.

9. Admin Dashboard Skill taxonomy editor with merge and synonym tools · evidence source configuration and weights · readiness model settings · drive management · recruiter grant management with expiry · verification queue · access-log review.

10. Reporting & Analytics Batch readiness distribution · **skill supply against drive demand – the single most valuable report in this project**, because it tells the training team what to run next term · gap frequency across the cohort · verification coverage · drive conversion funnel from eligible to registered to shortlisted to selected · training-effectiveness analysis · department comparison.

11. Notifications & Communication New drive matching profile · eligibility met or missed with reason · verification requested and completed · preparation checkpoint due · shortlist result · low-readiness nudge to student and mentor.

12. Third-Party Integrations ERP/SIS for academics, LMS for assessments, coding-assessment platforms of the HackerRank or Codility class by API or import, certification verification (P12 can supply this), email and SMS, SSO.

13. Expected Deliverables Per CES, plus a populated skill taxonomy for at least three target roles – software development, data, testing.

14. Development Milestones M0 discovery and taxonomy workshop (2w) · M1 taxonomy, evidence, verification (4w) · M2 profile and assessments (3w) · M3 readiness and gap analysis (4w) · M4 drives and preparation planning (3w) · M5 recruiter portal and analytics (3w) · M6 UAT, security review, production (2w).

15. Testing Per CES. **A dedicated authorisation test suite for the recruiter portal is a named deliverable.** End-to-end: evidence added, verified, profile updates, drive published, gap analysis, preparation plan, recruiter views shortlist.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. The recruiter portal deployed as a separate route group with independent rate limiting at the edge.

TRACK P · Python + Django + PostgreSQL. Per CES. The recruiter portal may share the process but must not share the session cookie domain or the throttle scope; document both settings.

17. Documentation Per CES, plus a documented **data-release policy** stating exactly which student fields a recruiter may see.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · MongoDB · search engines and taxonomy modelling · scoring and matching algorithms · data visualisation · strict RBAC and ABAC.

TRACK P · Python + Django + PostgreSQL. Python · Django and **DRF permission classes and throttling** · PostgreSQL full-text search · taxonomy modelling · scoring algorithms · Celery · INT252 for the dashboard.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. **One member strong on access control**, because the recruiter portal is the highest-risk surface here; one comfortable with search and taxonomy modelling. *Track P gaps:* DRF, Postgres full-text search, Celery.

20. Estimated Complexity and Semester Fit **Large.** 1,200–1,600 hours. **Two semesters for a team of 5**, or three parallel module teams (M1–M3 taxonomy and evidence · M4–M6 profile and readiness · M7–M10 drives, gaps and recruiter portal).

21. Suggested Student Team 5 students – tech lead · 2 backend, one owning authorisation · frontend and visualisation · QA and documentation.

22. Acceptance Criteria

- Skill profile assembled from at least four evidence types with verification status visible
- Eligibility and readiness computed and displayed independently
- Gap analysis produces ranked, specific, actionable gaps for any student against any drive
- Preparation plan generated with dated checkpoints
- Recruiter cannot access any student outside their approved drive – proven by negative tests
- All recruiter access logged and reviewable
- Batch readiness for 5,000 students computed as a background job in under 15 minutes
- Skill supply-against-demand report produced for a real drive

AI/ML potential: Substantial and well-founded – resume and project text to skill extraction with a language model under human verification, selection-probability modelling once two or more years of drive outcomes exist, and preparation-resource recommendation. Keep the human verification step permanently.

Commercialisation: The highest of any project in this portfolio. Placement readiness is a universal pain across Indian engineering education and there is no dominant incumbent.